МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ №1

з дисципліни **"Функціональне програмування"**

на тему **"ОСНОВИ ФУНКЦІОНАЛЬНОГО ПРОГРАМУВАННЯ У PYTHON"**

Варіант 13. Страхові поліси

Виконав:
Студент групи ФЕП-23
Маньківський Максим Володимирович

Перевірив:
проф. Свелеба С. А.

Львів-2026

Мета: зрозуміти поняття чистої функції та референтної прозорості, навчитися виявляти й ізолювати побічні ефекти, переписати імперативний фрагмент розрахунку страхових премій у функціональному стилі та параметризувати обчислення за допомогою `typing.Callable`.

Теоретичні відомості:

Чиста функція — це функція, яка детермінована (однаковий вхід завжди дає однаковий вихід), не змінює глобального стану та вхідних даних і не виконує введення-виведення. Вираз, побудований із чистих функцій, є референтно прозорим: його можна замінити на обчислене значення без зміни поведінки програми, що спрощує тестування, мемоізацію та міркування про код.

Побічними ефектами є I/O (`print`, файли, мережа), мутація вхідних структур, читання або зміна глобальних змінних, залежність від часу, випадковості чи оточення. Функціональний підхід полягає у відокремленні чистого ядра, яке лише перетворює дані, від тонкої оболонки ефектів, яка читає вхід і виводить результат. Залежності на кшталт годинника або генератора випадкових чисел передаються в ядро як функції (інжекція залежностей), а правила обчислень (коефіцієнт ризику, знижка за стаж, фільтр) — як значення типу `Callable`, що робить код компонувальним: будь-яку політику можна замінити, передавши іншу функцію.

Хід роботи:

1. За варіантом 13 (страхові поліси: базова премія, ризик, стаж клієнта, активність) написав стартовий імперативний фрагмент `starter_imperative.py`, що для активних полісів рахує кінцеву премію. У ньому є ті самі проблеми, що й у прикладі з методички: глобальні константи `RISK_RATES`, `DISCOUNT_PER_YEAR`,

`MAX_DISCOUNT`, виклик `print` усередині обчислень, мутація вхідних словників (`p["premium"] = …`) та змішування обчислення з I/O:

```python
TAX_RATE = 0.2  # глобальний стан
RISK_RATES = {"low": 0.9, "medium": 1.0, "high": 1.5}
DISCOUNT_PER_YEAR = 0.02
MAX_DISCOUNT = 0.3


def process_orders(
    orders: list[Any], min_total: float, discount: float
) -> dict[str, Any]:
    valid: list[dict[str, Any]] = []
    total_revenue = 0.0

    for o in orders:
        if not o["paid"]:
            continue
        print("Processing order:", o["id"])

        order_total = 0.0
        for it in o["items"]:
            order_total += it["price"] * it["qty"]
        if order_total < min_total:
            continue

        order_total = order_total * (1 - discount)
        order_total = order_total * (1 + TAX_RATE)

        o["total"] = order_total
        valid.append(o)
        total_revenue += order_total

    return {"count": len(valid), "revenue": total_revenue, "orders": valid}


def process_policies(policies: list[Any], min_premium: float) -> dict[str, Any]:
    valid: list[dict[str, Any]] = []
    total_premium = 0.0

    for p in policies:
        if not p["active"]:
            continue
        print("Processing policy:", p["id"])

        premium = p["base_premium"]
        if premium < min_premium:
            continue

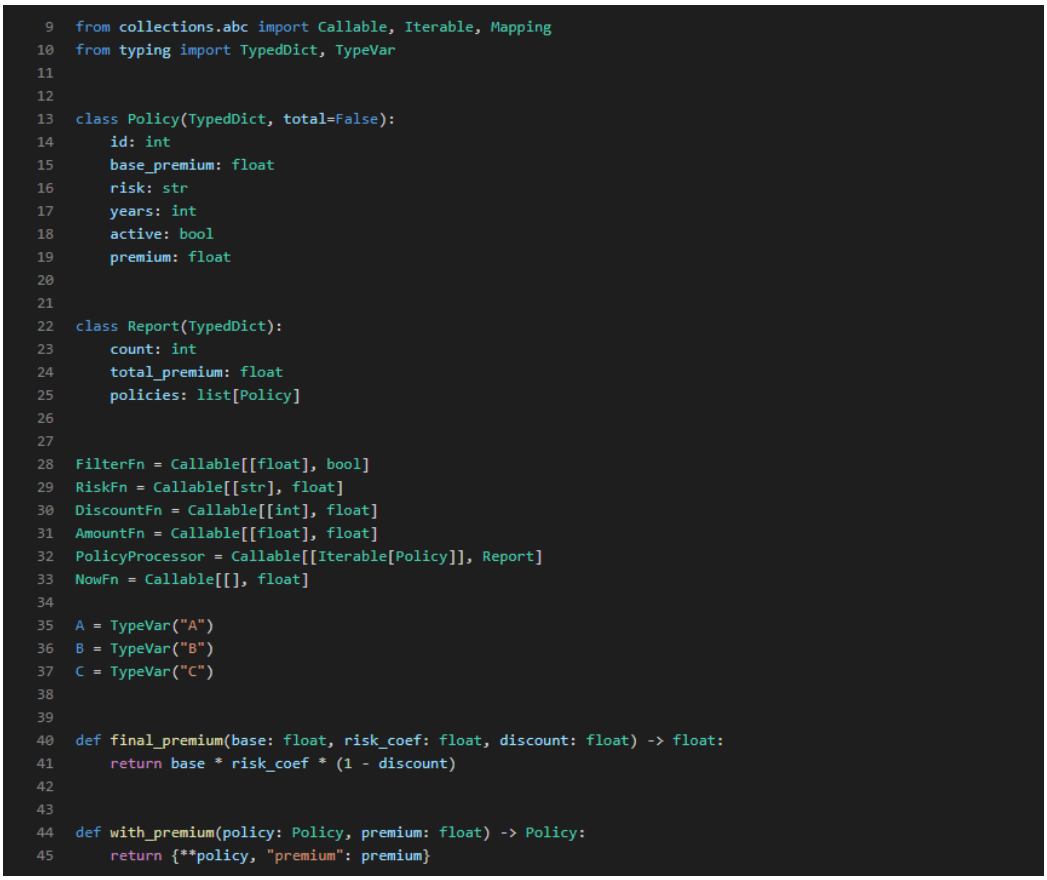
        premium = premium * RISK_RATES[p["risk"]]
        discount = p["years"] * DISCOUNT_PER_YEAR
        if discount > MAX_DISCOUNT:
            discount = MAX_DISCOUNT
        premium = premium * (1 - discount)

        p["premium"] = premium
        valid.append(p)
        total_premium += premium

    return {"count": len(valid), "total_premium": total_premium, "policies": valid}
```

2. У файлі `core.py` описав типи даних через `TypedDict` (`Policy`, `Report`), псевдоніми політик на основі `Callable` та дві базові чисті функції: `final_premium` обчислює премію як добуток базової премії, коефіцієнта ризику

та множника знижки, а `with_premium` повертає новий словник, не змінюючи вхідний:



```python
from collections.abc import Callable, Iterable, Mapping
from typing import TypedDict, TypeVar


class Policy(TypedDict, total=False):
    id: int
    base_premium: float
    risk: str
    years: int
    active: bool
    premium: float


class Report(TypedDict):
    count: int
    total_premium: float
    policies: list[Policy]


FilterFn = Callable[[float], bool]
RiskFn = Callable[[str], float]
DiscountFn = Callable[[int], float]
AmountFn = Callable[[float], float]
PolicyProcessor = Callable[[Iterable[Policy]], Report]
NowFn = Callable[[], float]

A = TypeVar("A")
B = TypeVar("B")
C = TypeVar("C")


def final_premium(base: float, risk_coef: float, discount: float) -> float:
    return base * risk_coef * (1 - discount)


def with_premium(policy: Policy, premium: float) -> Policy:
    return {**policy, "premium": premium}
```

3. Реалізував фабрику `make_policy_processor`, яка приймає три політики (`accept` — фільтр за базовою премією, `risk_factor` — коефіцієнт ризику, `loyalty_discount` — знижка за стаж) і повертає налаштований обробник. Обробник відбирає активні поліси через list comprehension, обчислює кінцеву премію і повертає новий результат. Функція `process_policies_pure` є окремим випадком: вона збирає обробник із фабрик політик `min_premium_filter`,

`table_risk` і `years_discount`, отримуючи всі параметри (таблицю ризиків, відсоток за рік стажу, максимальну знижку) явно:

```python
def make_policy_processor(
    *,
    accept: FilterFn,
    risk_factor: RiskFn,
    loyalty_discount: DiscountFn,
) -> PolicyProcessor:
    def process(policies: Iterable[Policy]) -> Report:
        qualified = [
            with_premium(
                p,
                final_premium(
                    p["base_premium"],
                    risk_factor(p["risk"]),
                    loyalty_discount(p["years"]),
                ),
            )
            for p in policies
            if p["active"] and accept(p["base_premium"])
        ]
        return {
            "count": len(qualified),
            "total_premium": sum(p["premium"] for p in qualified),
            "policies": qualified,
        }

    return process


def min_premium_filter(min_premium: float) -> FilterFn:
    return lambda base: base >= min_premium


def table_risk(rates: Mapping[str, float], default: float = 1.0) -> RiskFn:
    return lambda risk: rates.get(risk, default)


def years_discount(rate_per_year: float, max_discount: float) -> DiscountFn:
    return lambda years: min(years * rate_per_year, max_discount)


def process_policies_pure(
    policies: Iterable[Policy],
    *,
    min_premium: float,
    risk_rates: Mapping[str, float],
    discount_per_year: float,
    max_discount: float,
) -> Report:
    process = make_policy_processor(
        accept=min_premium_filter(min_premium),
        risk_factor=table_risk(risk_rates),
        loyalty_discount=years_discount(discount_per_year, max_discount),
    )
    return process(policies)
```

4. Додав функції з додаткових вправ: узагальнену композицію `compose`, фабрику множників `make_multiplier` та `stamp_total`, яка отримує годинник ззовні як `Callable`, тому залишається тестованою без залежності від реального часу:

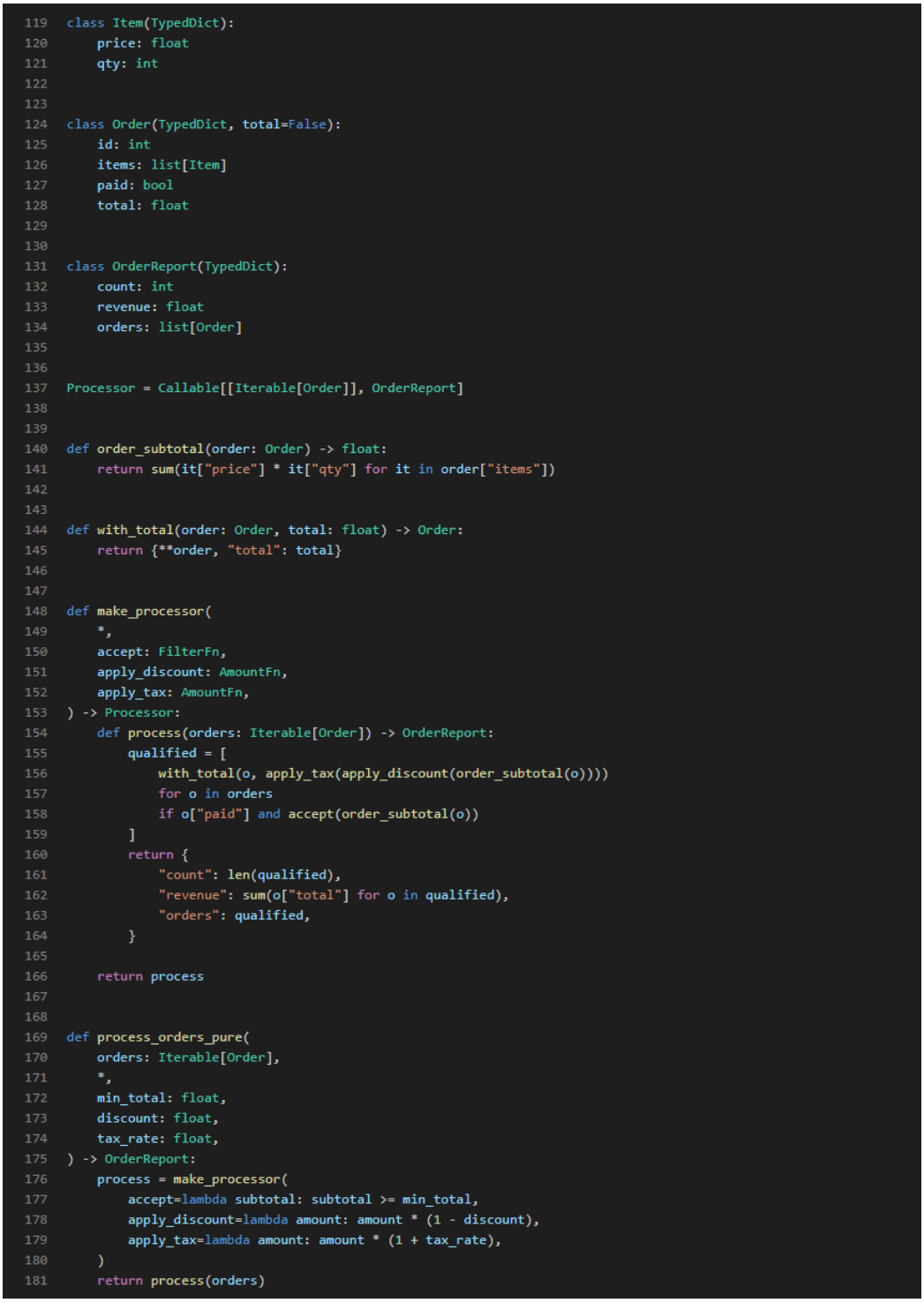
```python
def compose(f: Callable[[B], C], g: Callable[[A], B]) -> Callable[[A], C]:
    return lambda x: f(g(x))


def make_multiplier(k: int) -> Callable[[int], int]:
    return lambda x: x * k


def stamp_total(total: float, now: NowFn) -> tuple[float, float]:
    return total, now()

```

5. Базове завдання з методички (обробка замовлень) також переписав у функціональному стилі в тому ж `core.py`: `order_subtotal`, `with_total`, фабрика `make_processor` з політиками `accept`, `apply_discount`, `apply_tax` та `process_orders_pure` з явними параметрами `min_total`, `discount`, `tax_rate`:



```python
class Item(TypedDict):
    price: float
    qty: int


class Order(TypedDict, total=False):
    id: int
    items: list[Item]
    paid: bool
    total: float


class OrderReport(TypedDict):
    count: int
    revenue: float
    orders: list[Order]


Processor = Callable[[Iterable[Order]], OrderReport]


def order_subtotal(order: Order) -> float:
    return sum(it["price"] * it["qty"] for it in order["items"])


def with_total(order: Order, total: float) -> Order:
    return {**order, "total": total}


def make_processor(
    *,
    accept: FilterFn,
    apply_discount: AmountFn,
    apply_tax: AmountFn,
) -> Processor:
    def process(orders: Iterable[Order]) -> OrderReport:
        qualified = [
            with_total(o, apply_tax(apply_discount(order_subtotal(o))))
            for o in orders
            if o["paid"] and accept(order_subtotal(o))
        ]
        return {
            "count": len(qualified),
            "revenue": sum(o["total"] for o in qualified),
            "orders": qualified,
        }

    return process


def process_orders_pure(
    orders: Iterable[Order],
    *,
    min_total: float,
    discount: float,
    tax_rate: float,
) -> OrderReport:
    process = make_processor(
        accept=lambda subtotal: subtotal >= min_total,
        apply_discount=lambda amount: amount * (1 - discount),
        apply_tax=lambda amount: amount * (1 + tax_rate),
    )
    return process(orders)
```

6. Усі побічні ефекти виніс в оболонку `app.py`: читання полісів із JSON-файлу, функцію `render_report`, що лише перетворює чистий результат на текст,

функцію `run` для базового завдання, друк та передачу `time.time` у ядро:

```python
from core import (
    Order,
    Policy,
    Report,
    process_orders_pure,
    process_policies_pure,
    stamp_total,
)

RISK_RATES = {"low": 0.9, "medium": 1.0, "high": 1.5}

SAMPLE: list[Policy] = [
    {"id": 1, "base_premium": 1000.0, "risk": "low", "years": 5, "active": True},
    {"id": 2, "base_premium": 300.0, "risk": "high", "years": 0, "active": True},
    {"id": 3, "base_premium": 2000.0, "risk": "medium", "years": 20, "active": False},
    {"id": 4, "base_premium": 1500.0, "risk": "high", "years": 20, "active": True},
]


def load_policies(path: Path) -> list[Policy]:
    return list(json.loads(path.read_text(encoding="utf-8")))


def render_report(result: Report) -> str:
    lines = [
        f"Policy: {p['id']} premium: {p['premium']:.2f}" for p in result["policies"]
    ]
    return "\n".join(
        [*lines, f"Count: {result['count']}", f"Total: {result['total_premium']:.2f}"]
    )


def run(orders: list[Order]) -> None:
    """Оболонка базового завдання: друкує чистий результат process_orders_pure."""
    result = process_orders_pure(orders, min_total=100, discount=0.1, tax_rate=0.2)
    for o in result["orders"]:
        print("Processed:", o["id"], "total:", o["total"])
    print("Revenue:", result["revenue"])


def main(argv: list[str]) -> None:
    policies = load_policies(Path(argv[1])) if len(argv) > 1 else SAMPLE
    result = process_policies_pure(
        policies,
        min_premium=500,
        risk_rates=RISK_RATES,
        discount_per_year=0.02,
        max_discount=0.3,
    )
    print(render_report(result))
    total, stamp = stamp_total(result["total_premium"], time.time)
    print(f"Stamped: {total:.2f} at {time.strftime('%H:%M:%S', time.localtime(stamp))}")


if __name__ == "__main__":
    main(sys.argv)
```

7. Написав pytest-тести у `tests/test_core.py`: референтна прозорість, відсутність мутації вхідних даних, еквівалентність імперативній версії та

правильність фільтрації:

```python
RATES = {"low": 0.9, "medium": 1.0, "high": 1.5}


def run(policies: list[Policy]) -> Report:
    return process_policies_pure(
        policies,
        min_premium=500.0,
        risk_rates=RATES,
        discount_per_year=0.02,
        max_discount=0.3,
    )


def test_referential_transparency(sample_policies: list[Policy]) -> None:
    r1 = run(sample_policies)
    r2 = run(sample_policies)
    assert r1 == r2


def test_no_mutation(sample_policies: list[Policy]) -> None:
    original = deepcopy(sample_policies)
    process_policies_pure(
        sample_policies,
        min_premium=0,
        risk_rates=RATES,
        discount_per_year=0.0,
        max_discount=0.0,
    )
    assert sample_policies == original


def test_matches_imperative(sample_policies: list[Policy]) -> None:
    expected = process_policies(deepcopy(sample_policies), 500)
    assert run(sample_policies) == expected


def test_filters_inactive_and_small(sample_policies: list[Policy]) -> None:
    result = run(sample_policies)
    assert [p["id"] for p in result["policies"]] == [1, 4, 5]
    assert result["count"] == 3
    assert result["total_premium"] == pytest.approx(
        1000 * 0.9 * 0.9 + 1500 * 1.5 * 0.7 + 500 * 1.0 * 0.98
    )

```

8. Додав тести на Callable-політики (`make_policy_processor` з лямбдами), фабрики політик, `compose` та `make_multiplier`, інжекцію годинника й текст звіту,

а в `tests/test_orders.py` — мінімальні тести з методички для базового завдання:

```python
def test_final_premium() -> None:
    assert final_premium(1000.0, 1.5, 0.1) == pytest.approx(1350.0)
    assert final_premium(0.0, 2.0, 0.5) == 0.0


def test_with_premium_returns_new_dict() -> None:
    policy: Policy = {
        "id": 1,
        "base_premium": 100.0,
        "risk": "low",
        "years": 0,
        "active": True,
    }
    new = with_premium(policy, 42.0)
    assert new == {**policy, "premium": 42.0}
    assert "premium" not in policy
    assert new is not policy


def test_callable_policies(sample_policies: list[Policy]) -> None:
    process = make_policy_processor(
        accept=lambda base: base >= 1000,
        risk_factor=lambda risk: 2.0 if risk == "high" else 1.0,
        loyalty_discount=lambda years: 0.5 if years >= 10 else 0.0,
    )
    result = process(sample_policies)
    assert [p["id"] for p in result["policies"]] == [1, 4]
    assert result["total_premium"] == pytest.approx(1000 * 1.0 + 1500 * 2.0 * 0.5)


def test_policy_factories() -> None:
    assert min_premium_filter(500)(499.9) is False
    assert min_premium_filter(500)(500) is True
    assert table_risk(RATES)("high") == 1.5
    assert table_risk(RATES, default=1.0)("unknown") == 1.0
    assert years_discount(0.02, 0.3)(5) == pytest.approx(0.1)
    assert years_discount(0.02, 0.3)(50) == 0.3


def test_compose_and_multiplier() -> None:
    discount_for = compose(years_discount(0.02, 0.3), make_multiplier(2))
    assert discount_for(5) == pytest.approx(0.2)
    assert make_multiplier(3)(7) == 21
    assert compose(make_multiplier(2), make_multiplier(5))(1) == 10


def test_stamp_total_uses_injected_clock() -> None:
    assert stamp_total(10.0, lambda: 123.0) == (10.0, 123.0)


def test_render_report(sample_policies: list[Policy]) -> None:
    text = render_report(run(sample_policies))
    assert text.splitlines() == [
        "Policy: 1 premium: 810.00",
        "Policy: 4 premium: 1575.00",
        "Policy: 5 premium: 490.00",
        "Count: 3",
        "Total: 2875.00",
    ]
```

```python


def test_referential_transparency(sample_orders: list[Order]) -> None:
    r1 = process_orders_pure(sample_orders, **ARGS)
    r2 = process_orders_pure(sample_orders, **ARGS)
    assert r1 == r2


def test_no_mutation(sample_orders: list[Order]) -> None:
    original = deepcopy(sample_orders)
    process_orders_pure(sample_orders, min_total=0, discount=0.0, tax_rate=0.0)
    assert sample_orders == original


def test_matches_imperative(sample_orders: list[Order]) -> None:
    expected = process_orders(deepcopy(sample_orders), 100, 0.1)
    assert process_orders_pure(sample_orders, **ARGS) == expected


def test_callable_policies(sample_orders: list[Order]) -> None:
    process = make_processor(
        accept=lambda s: s >= 50,
        apply_discount=lambda s: s * 0.9,
        apply_tax=lambda s: s * 1.2,
    )
    result = process(sample_orders)
    assert [o["id"] for o in result["orders"]] == [1, 4]
    assert result["revenue"] == pytest.approx((150 + 111) * 0.9 * 1.2)


def test_subtotal_and_with_total() -> None:
    order: Order = {"id": 1, "items": [{"price": 2.5, "qty": 4}], "paid": True}
    assert order_subtotal(order) == 10.0
    new = with_total(order, 42.0)
    assert new["total"] == 42.0
    assert "total" not in order
```

9. Запустив оболонку на вбудованому прикладі: з чотирьох полісів відібрано два активних із базовою премією не менше 500, до них застосовано коефіцієнт ризику (0.9 / 1.0 / 1.5) і знижку 2 % за рік стажу, але не більше 30 %:

```
C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab01> python app.py
Policy: 1 premium: 810.00
Policy: 4 premium: 1575.00
Count: 2
Total: 2385.00
Stamped: 2385.00 at 13:30:29
```

10. Перевірив проєкт автоматичними інструментами: усі 16 тестів pytest проходять, `mypy --strict` не знаходить помилок типів, `ruff` та `black` не мають

зауважень до стилю:

```
C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab01> python -m pytest -v
============================= test session starts =============================
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\maksi\AppData\Local\Python\pythoncore-3.14-64\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\maksi\Desktop\labs\FunctionalProgramming
configfile: pyproject.toml
plugins: platformdirs-4.12.3
collecting ... collected 16 items

tests\test_core.py::test_referential_transparency PASSED                 [  6%]
tests\test_core.py::test_no_mutation PASSED                              [ 12%]
tests\test_core.py::test_matches_imperative PASSED                       [ 18%]
tests\test_core.py::test_filters_inactive_and_small PASSED               [ 25%]
tests\test_core.py::test_final_premium PASSED                            [ 31%]
tests\test_core.py::test_with_premium_returns_new_dict PASSED            [ 37%]
tests\test_core.py::test_callable_policies PASSED                        [ 43%]
tests\test_core.py::test_policy_factories PASSED                         [ 50%]
tests\test_core.py::test_compose_and_multiplier PASSED                   [ 56%]
tests\test_core.py::test_stamp_total_uses_injected_clock PASSED          [ 62%]
tests\test_core.py::test_render_report PASSED                            [ 68%]
tests\test_orders.py::test_referential_transparency PASSED               [ 75%]
tests\test_orders.py::test_no_mutation PASSED                            [ 81%]
tests\test_orders.py::test_matches_imperative PASSED                     [ 87%]
tests\test_orders.py::test_callable_policies PASSED                      [ 93%]
tests\test_orders.py::test_subtotal_and_with_total PASSED                [100%]

============================== 16 passed in 0.04s ==============================

C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab01> python -m mypy .
Success: no issues found in 6 source files

C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab01> python -m ruff check .
All checks passed!

C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab01> python -m black --check .
All done! ✨ 🍰 ✨
6 files would be left unchanged.
```

Висновок: Під час виконання лабораторної роботи я розібрав поняття чистої функції, референтної прозорості та побічних ефектів і переписав імперативний розрахунок страхових премій (варіант 13) у функціональному стилі: прибрав глобальний стан, мутації вхідних даних і `print` із ядра, виніс I/O в окрему оболонку, а політики коефіцієнта ризику, знижки за стаж та фільтрації передав через `typing.Callable` у фабрику `make_policy_processor`. Тести на референтну прозорість і відсутність мутацій, а також перевірки mypy, ruff і black підтвердили коректність і чистоту реалізації.